

¿Cuáles son los 4 pilares de la programación orientada a objetos y qué aporta cada uno?

Encapsulamiento

Protege la información, oculta los detalles internos de un objeto y restringe el acceso a sus datos mediante modificadores de acceso. Se usan métodos getters y setters para controlar el acceso a los atributos

Ejemplo

```
class Persona:
    def __init__(self,nombre,edad):
        self.__nombre = nombre
        self.__edad = edad

    def get_nombre(self):
        return self.__nombre

    def set_nombre(self, nuevo_nombre):
        self.__nombre = nuevo_nombre

    def get_edad(self):
        return self.__edad

persona1 = Persona("Ana", 30)
print(persona1.get_nombre())
nuevo_nombre = input("Ingrese el nuevo nombre: ")
persona1.set_nombre(nuevo_nombre)
print(persona1.get_nombre())
print(persona1.get_edad())
```

Abstracción

Consiste en representar las características esenciales de un objeto. Se logra mediante clases y objetos, que modelan entidades del mundo real. Por ejemplo un objeto Coche puede tener atributos como marca, modelo y métodos como acelerar() o frenar() sin preocuparse por cómo funciona internamente el motor que podría ser otra clase.

Ejemplo

```
class Coche:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def acelerar(self):
        print(f"{self.marca} {self.modelo} esta acelerando.")

mi_coche = Coche("Fiat", "Mobi")
mi_coche.acelerar()
```

Herencia

Permite que una clase(subclase) hereda atributos y métodos de otra clase (superclase/clase padre) promoviendo la reutilización de código.

Por ejemplo la clase Perro y la clase Gato pueden heredar características de la clase Animal.

```
class Animal:
    def __init__(self,nombre):
        self.nombre = nombre

    def hacer_sonido(self):
        print('Sonido genérico')

class Perro(Animal):
    def hacer_sonido(self):
        print('Guau!')

mi_animal = Animal('Generico')
mi_animal.hacer_sonido()

mi_perro = Perro('Fido')
mi_perro.hacer_sonido()
```

Polimorfismo

Permite que un objeto se comporte de múltiples formas, ya sea mediante Sobreescritura de métodos (overdrive) = Redefinir un método de la superclase en la subclase

o por medio de sobrecarga de métodos (Overload) = Métodos con el mismo nombre pero diferentes parámetros

```
class Gato(Animal):
    def hacer_sonido(self):
        print('Miau')

def escuchar_sonido(animal):
    animal.hacer_sonido()

mis_animales = [Perro('Fido'), Gato('Michi'), Animal('Generico')]
for animal in mis_animales:
    escuchar_sonido(animal)
```

La diferencia entre encapsulamiento y abstracción es que: Encapsulamiento oculta los detalles internos de un objeto y restringe el acceso directo a sus datos, exponiendo solo lo necesario mediante métodos públicos. Y la abstracción consiste en simplificar la complejidad mostrando solo las características esenciales de un objeto y ocultando lo irrelevante. Define qué hace un objeto, no cómo lo hace.

El pilar que permite a las subclases sobrescribir métodos es el polimorfismo.

```
"""
```

Ejercicio:

Define una jerarquía simple para vehículos con al menos una clase base y dos clases hijas.

Cada clase hija debe tener un método propio sobrescrito que imprima información

diferente. Crea una función que reciba un vehículo y llame a ese método.

```
"""
```

```
class Vehiculo:
```

```
    def __init__(self, ruedas, velocidad_maxima, color):
```

```
        self.ruedas = ruedas
```

```
        self.velocidad_maxima = velocidad_maxima
```

```
        self.color = color
```

```
    def acelerar(self):
```

```
        print(f"El vehiculo de color {self.color} con {self.ruedas}  
ruedas esta acelerando y llego a una velocidad maxima de:  
{self.velocidad_maxima} Km/h")
```

```
    def probar_aceleracion(vehiculo):
```

```
        vehiculo.acelerar()
```

```
class Moto(Vehiculo):
```

```
    def acelerar(self):
```

```
        print(f"La moto de color {self.color} con {self.ruedas}  
ruedas esta acelerando y llego a una velocidad maxima de  
{self.velocidad_maxima} Km/h ")
```

```
class Auto(Vehiculo):
```

```
    def acelerar(self):
```

```
        print(f"El auto de color {self.color} con {self.ruedas}  
ruedas esta acelerando y llego a una velocidad maxima de  
{self.velocidad_maxima} Km/h")
```

```
moto1 = Moto(2, 80, "Roja")
```

```
vehiculo1 = Vehiculo(8, 140, "Negro")  
auto1 = Auto(4, 120, "Azul")
```

```
moto1.probar_aceleracion()  
vehiculo1.probar_aceleracion()  
auto1.probar_aceleracion()
```